

17.6

Autoscaling

HPA, VPA, Cluster Autoscaler & KEDA

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

Three Dimensions of Scaling

Kubernetes can scale in three ways: more pod replicas (HPA), bigger pods (VPA), and more nodes (Cluster Autoscaler). Together they handle any load increase automatically.

■ **Simple Analogy:** HPA = hire more workers for peak season. VPA = give each worker a bigger desk. Cluster Autoscaler = rent more office space when you run out of desks.

HPA — Horizontal Pod Autoscaler

HPA watches a metric (usually CPU or memory) and adds/removes pods automatically. Every 15 seconds it checks: is actual metric above/below target?

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: java-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: java-api
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70 # target 70% CPU
    # Formula: desiredReplicas = ceil(current * actualCPU / 70)
```

HPA Scaling Formula

Simple example: if you have 3 pods running at 90% CPU and target is 70%:

$\text{desiredReplicas} = \text{ceil}(3 \times 90\% / 70\%) = \text{ceil}(3.86) = \mathbf{4 \text{ pods}}$

K8s adds 1 pod. After warmup, if CPU drops to 60%:

$\text{desiredReplicas} = \text{ceil}(4 \times 60\% / 70\%) = \text{ceil}(3.43) = \mathbf{4 \text{ pods}}$ (no change — within tolerance)

VPA — Vertical Pod Autoscaler

VPA adjusts CPU/memory requests for pods based on actual usage. Currently requires pod restart to apply new resource values. Useful for: long-running batch jobs, services with unpredictable memory needs.

Cluster Autoscaler

When HPA tries to add pods but no node has enough capacity, Cluster Autoscaler provisions a new node from your cloud provider automatically. When nodes are underutilized, it drains and terminates them to save cost.

KEDA — Event-Driven Autoscaling

KEDA extends HPA to scale on ANY metric: Kafka lag, SQS queue depth, Prometheus metrics, or HTTP requests. Crucially, it can scale TO ZERO when there is no work to do.

```
# KEDA: scale based on Kafka consumer lag
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
spec:
  scaleTargetRef:
    name: order-consumer
  minReplicaCount: 0 # scale to ZERO when no messages
  maxReplicaCount: 50
  triggers:
    - type: kafka
  metadata:
    topic: order.events
    consumerGroup: order-consumer-group
    lagThreshold: "100" # add 1 pod per 100 unprocessed messages
```

■ **Real-World Example:** Zalando uses KEDA in production. During sale events, Kafka consumer pods scale from 0 to 200+ within minutes based on topic lag. Off-peak, they scale back to 0 saving ~70% compute cost. HPA alone can't do this — it needs at least 1 replica always.